



FASE 2 · BLOCKCHAIN

Mini Blockchain DApp

Um contrato inteligente na rede local do Hardhat, operado por uma interface web, sem terminal.

O que vou mostrar

#	Parte	Conteúdo	Tempo
1	Da fase 1 para a fase 2	Python virou Solidity	2 min
2	O contrato	Funções de leitura e escrita	3 min
3	Hardhat e testes	Rede local, deploy, 8 testes	1 min
4	Demonstração ao vivo	Minerar, enviar, erros, adulteração	4 min
5	Fechamento	Objetivos do enunciado atendidos	1 min

Mesma lógica, outro lugar para rodar

Fase 1 • Python	Fase 2 • Solidity
compute_hash() com SHA-256	computeHash() com keccak256
mine() procura o nonce	O navegador minera; o contrato só confere
add_block()	addBlock(dados, nonce)
is_valid(): 3 checagens	isValid(): as mesmas 3 checagens e mensagens
chain[1].data["valor"] = 5000 funcionava	Impossível: não existe função de edição

A IDEIA CENTRAL

Difícil de achar, fácil de verificar.

0x00003ec0b7780f2c6c2e5c28b01f130e...

Navegador minera

Cerca de 65 mil tentativas por bloco na dificuldade 4.

Contrato confere

Um único hash: barato em gas.

A interface fala com o contrato pela rede local



Escrita: 2 funções que viram transação

`addBlock(dados, nonce)`

Calcula o hash com o nonce enviado, exige os zeros da dificuldade e grava o bloco ligado ao anterior.

Emite o evento BlockMined.

Sem prova de trabalho, a transação é revertida.

`setDifficulty(n)`

Só o dono (a conta do deploy) muda quantos zeros o hash precisa ter, até 6. Emite DifficultyChanged.

Cada zero a mais multiplica o trabalho por 16.

Leitura: 7 funções grátis, só consultam

Função	O que devolve
getChain()	A cadeia inteira; é o que a tela de blocos mostra
getBlock(i)	Um bloco pelo índice
totalBlocks()	Quantos blocos existem (= índice do próximo)
lastHash()	Hash do último bloco, o prevHash do próximo
isValid()	Revalida tudo: hash, elo e prova de trabalho
computeHash(...)	Hash de um bloco sem gravar; usado na simulação
meetsDifficulty(h, z)	Se o hash começa com z zeros hexadecimais

Três decisões de projeto

01

O minerador entra no hash

Ninguém copia o nonce que outra conta minerou e envia como se fosse seu.

02

O horário fica fora do hash

O minerador não sabe antes em que bloco a transação entra; o horário vem de `block.timestamp`.

03

Cada bloco guarda sua dificuldade

Mudar a dificuldade depois não invalida os blocos antigos.

Um clique no iniciar.bat sobe tudo

1 • Sobe a rede local

`npx hardhat node · chainId 31337 · 10 contas com 10.000 ETH de teste`

2 • Implanta o contrato

`deploy.js` grava os blocos da fase 1 (Alice → Bob 50, Bob → Carol 20) e o `contract.json`

3 • Abre a interface

`server.js` em `localhost:3000`

8/8

testes automatizados
passando

`npm test`

Do caso normal aos erros, terminando na adulteração

1

Conexão e leitura

Selo verde, cartões e a cadeia deslizando

2

Minerar e enviar

Barra de progresso e etapas até Confirmada

3

Trocar de conta

Conta #3 envia; a cadeia mostra o minerador

4

Sem minerar

Falhou: "Prova de trabalho inválida"

5

Permissão de dono

Só a Conta #0 muda a dificuldade, em Configurações

6

Simular adulteração

50 vira 5000: blocos vermelhos, a integridade segue Íntegra

IMUTABILIDADE

A adulteração só existe na tela.

Fase 1 · Python

```
chain[1].data["valor"] = 5000
```

Funcionava: a lista mudava na memória.

Fase 2 · Contrato

```
isValid() → true
```

Não existe função para editar blocos.

Os 4 objetivos do enunciado, atendidos

Objetivo	Como foi atendido
Integrar front-end e blockchain	HTML/JS com ethers.js v6, direto no Hardhat ou via MetaMask
Implantar na rede local do Hardhat	npx hardhat node + deploy.js, chainId 31337
Executar leitura e escrita	Leitura: getChain, isValid, computeHash. Escrita: addBlock, setDifficulty
Exibir resultados e estados	Minerando → Aguardando assinatura → Pendente → Confirmada ou Falhou, com hash, bloco, gas e motivo

O usuário usa o contrato inteiro sem abrir o terminal; as regras ficam no contrato.



Muito obrigado !